

Homework 1

Numerical Solutions to Partial Differential Equations

COLORADO SCHOOL OF MINES
DEPARTMENT OF APPLIED MATHEMATICS AND STATISTICS

AUTHOR INFORMATION
Hugh Scribner¹

¹Computational and Applied Mathematics

Contents

1	Problem 1	2
2	Problem 2	3
3	Problem 3	4

Disclaimer:

Claude was used to help prepare the results presented in this report. While all code used in the finite difference Solvers was written by a human, Claude was used to help plot and process data.

1 Problem 1

Problem Statement:

Simulate the stokes equations using a finite difference method on a staggered grid.
Provide an image of the magnitude of the velocity vector.

Solution: Below is a figure of the magnitude of the velocity as predicted by the finite difference solver on a 100x100 grid.

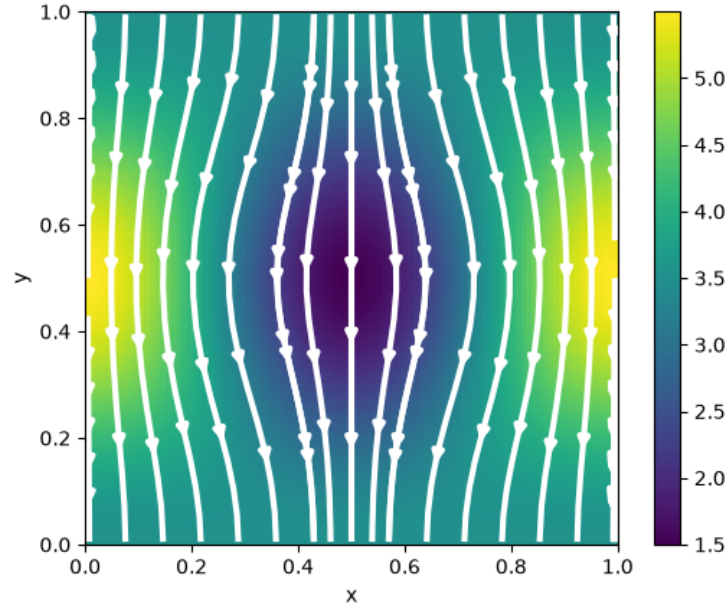


Figure 1: Magnitude of fluid velocity, $\vec{u}$, as predicted by a finite difference stokes solver.

Comparing each of the components of the numerical stokes solution, u , v , and p to their analytical counter parts, it can be seen that the maximum error in the solution is $\approx 6e - 4$ in the figure below. This is an indication that the solver is converging to an accurate solution.

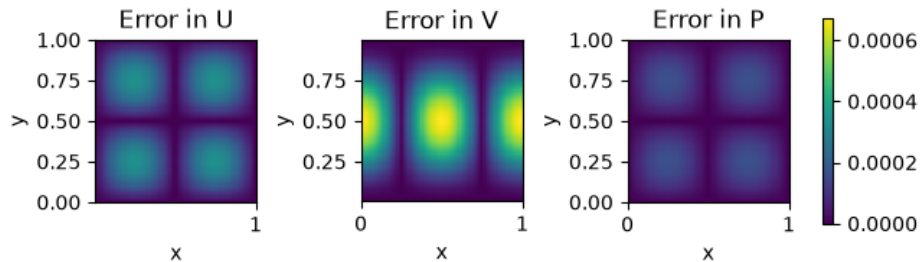


Figure 2: Absolute error between numerical approximations and analytical u , v and p solutions. Error was calculated using $E_{abs} = |u_{an} - u_{num}|$

2 Problem 2

Problem Statement:

Perform a convergence analysis on the solver you wrote in the previous problem. Show it has $\mathcal{O}(\Delta x^2)$ convergence.

Solution: The small absolute error in a refined grid is a good indicator of convergence, but to have confidence in the finite difference solver, it needs to show $\mathcal{O}(\Delta x^2)$ order convergence. Running the solver for a variety of grid sizes and plotting the L_2 error between the analytical solution and the exact solution, the finite difference solver demonstrates just that in each of u , v and p .

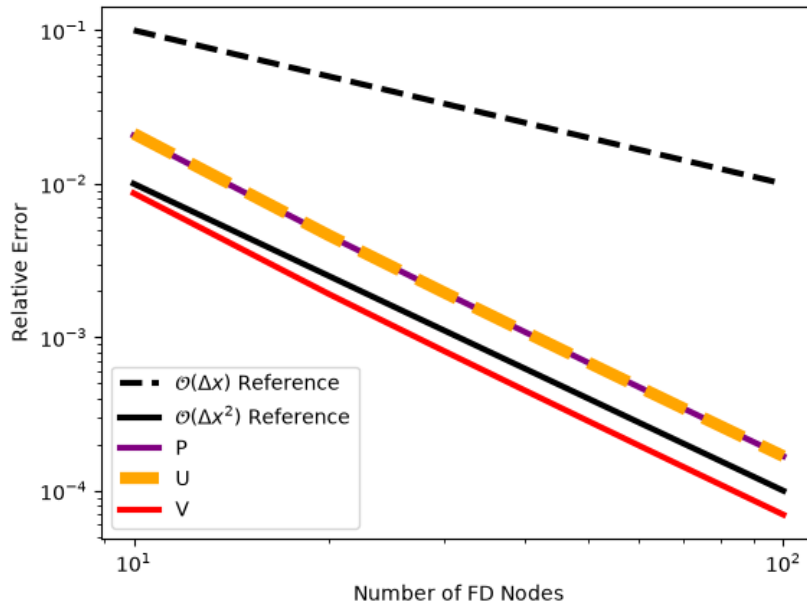


Figure 3: Relative error in finite difference approximations follow second order convergence.

3 Problem 3

Problem Statement:

While the problem in this homework assignment is small in scale, the underlying pipeline is the same you could use to approach a larger problem. However, this current implementation stalls out and takes tens of minutes to run on refined grids with more than 500 nodes. Below, I profile my code to determine where potential bottlenecks may apply.

Solution:

The main solver runs in two stages: system assembly and solution to the linear system. As problems grow larger, we know that the RAM requirements for this problem will sky rocket, but which part of the script takes the most time?

The Stokes solver was ran on an increasingly refined mesh and for several trials and a profile of each execution was captured using python's cprofiler package. The Assembly of the A matrix was run in two stages - buildLaplacian and BuildDeriv. Below are the findings on how the time complexity of each method scaled.

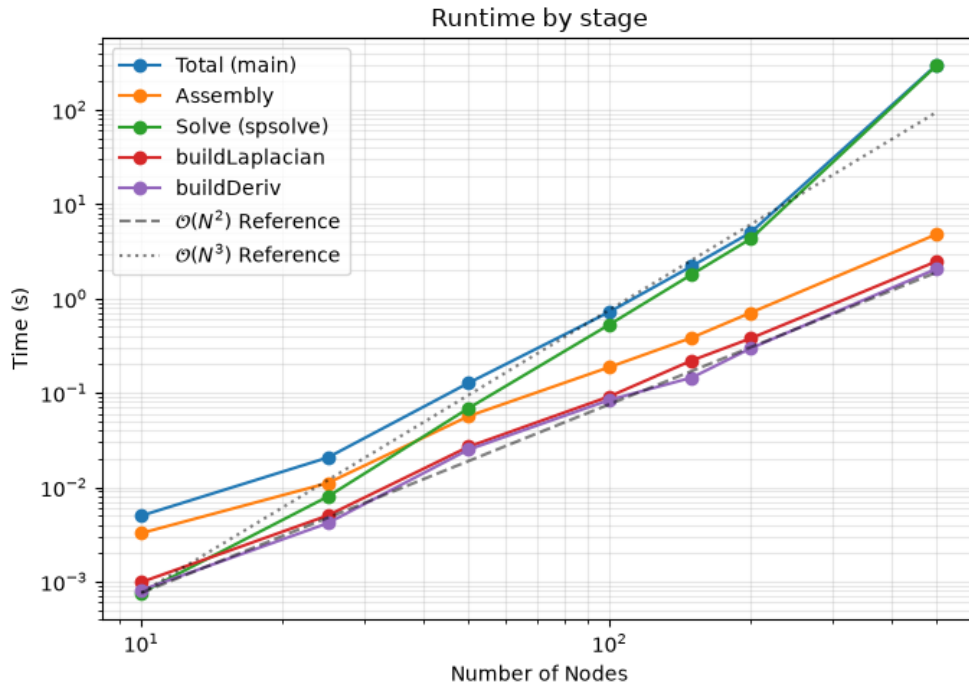


Figure 4: Above is a plot of the the run time of each stage in the stokes pipeline. Notice how the Solve grows in cubic time, while constructing the linear system completes in quadratic.

Figure 4 shows that the solve completes in cubic time while the system assembly completes in quadratic. This means that to speed up the solver, we would want to focus on that part of the code - perhaps by implementing a method like GMRES.